PRG Référence C++

v. 25.10.2026

Préprocesseur

`#include <iostream>`	Insère un fichier d'en-tête standard
`#include "monfichier.hpp"`	Insère un fichier d'en-tête local

`#ifndef NOM_HPP`
`#define NOM_HPP`
`/* … */`
`#endif`
	Empêche l'inclusion multiple d'un fichier d'en-tête : le code n'est inclus que si `NOM_HPP` n'est pas encore défini

`#pragma once`	Inclusion unique, non standard
`#define N 42`	Macro
`#undef N`	Supprime une macro
`#if / #elif / #else / #endif`	Compilation conditionnelle

Constantes littérales

`255, 0377, 0xff`	Entiers (décimal, octal, hexadécimal)
`123L, 234UL, 345LL, 456ull`	Entiers (`unsigned`) `long` et `long long`
`123.0f, 123.0, 1.23e2L`	Réels (`float`, `double`, `long double`)
`true, false`	Constantes booléennes (1 et 0)
`'a', '\141', '\x61'`	Caractères (littéral, octal, hexadécimal)
`'\n', '\\', '\'', '\"'`	Saut de ligne, backslash, apostrophe, guillemet
`"hello" "world"`	Chaînes concaténées, de type `const char[11]`
`"hello"s`	Chaîne littérale de type `std::string` (nécessite `using namespace std;`)

`"hello"` est en réalité `const char t[] { 'h', 'e', 'l', 'l', 'o', '\0' };`

<numbers>

`numbers::e; numbers::e_v<T>;`	Constante e de type `double` ; e pour le type T
`log2e, log10e`	Logarithme de e en base 2, en base 10
`pi, inv_pi, inv_sqrtpi`	π, $1/\pi$, $\pi^{-1/2}$
`sqrt2, sqrt3, inv_sqrt3`	$2^{1/2}$, $3^{1/2}$, $3^{-1/2}$

Déclarations

`int x;`	Déclare x entier
`int x = 42;  int x(42);  int x{42};`	Déclare et initialise x entier
`signed char a; unsigned char b;`	Entier sur 8 bits : [-128, 127] et [0, 255]
`char a = 'A';`	Caractère sur 8 bits, ASCII jusqu'à 127
`short a; unsigned short b;`	≥ 16 bits : [-32768, 32767] et [0, 65535]
`int a; unsigned b;`	≥ 16 bits, typ. 32 : [-2E9, 2E9] et [0, 4E9]
`long a; unsigned long b;`	≥ 32 bits, 32 ou 64 selon l'OS
`long long a; unsigned long long b;`	≥ 64 bits : [-9E18, 9E18] et [0, 1.8E19]
`float a; double b; long double c;`	Réels sur 32 et 64 bits (23 et 52 bits de mantisse) ; `long double` : 64, 80 ou 128 bits selon la plateforme
`bool a = true;`	Booléen. `false` vaut 0, `true` vaut 1 en `int`
`int a, b, c = 3, d;`	Déclaration multiple
`int a[10];`	Tableau de 10 entiers, non initialisé
`int a[] = {1, 2, 3};`	Tableau de 3 entiers initialisé
`int a[5] = {1, 2};`	Tableau de 5 `int` initialisé à {1, 2, 0, 0, 0}
`const int a = 0; int const b = 1;`	Constantes : doivent être initialisées, ne peuvent pas être affectées
`int& r = x;`	r est une référence à x (synonyme)
`const int& r1 = a; int const& r2 = a;`	Références constantes : a n'est pas modifiable via r1 ou r2
`enum Couleur {rouge, vert, bleu};`	Définit le type énuméré Couleur (style C)
`enum Couleur a; Couleur b;`	Variables de type Couleur
`enum class VAlign {up, down};`	Définit le type énuméré VAlign (C++11)
`VAlign a = VAlign::up;`	Les valeurs d'une `enum class` sont qualifiées
`using Entier = int;`	Entier est un synonyme du type `int`

Classes de stockage

`int x;`	Globale si hors de tout {}. Initialisée à 0
`{ int x; }`	Locale au bloc {}. Non initialisée, n'existe que pendant l'exécution du bloc
`{ static int x; static int y = a; }`	Durée de vie globale, initialisée (à 0 ou à a) au premier passage
`static int x;`	Globale, visible seulement dans ce fichier
`extern int x;`	Déclarée ici, doit être définie ailleurs

Espaces de noms

`namespace N { void f() {} }`	Place f dans l'espace de noms N
`N::f();`	Utilise f de l'espace de noms N
`using namespace N;`	Rend f visible sans `N::`

Instructions

`x = y;`	Affectation
`int x;`	Déclaration
`;`	Instruction vide
`{ int x; a; }`	Un bloc équivaut à une instruction

`if (x) a;`
`else if (y) b;`
`else c;`
	Si x est vrai, exécuter a
	Sinon, si y est vrai, exécuter b
	Sinon (ni x ni y), exécuter c (optionnel)

`while (x) a;`
`for (x; y; z) a;`
`do a; while (x);`
	Répéter 0 fois ou plus, tant que x est vrai
	Équivalent à `{ x; while (y) { a; z; } }` sauf si a contient `continue`
	Équivalent à `a; while (x) a;`

`switch (x) {`
`  case X1: a;`
`  case X2: b;`
`  default: c;`
`}`
	x de type entier, `char` ou `enum`
	Si x == X1 (expression constante), saute ici
	Sinon, si x == X2, saute ici
	Sinon, saute ici (optionnel)
	Sans `break`, l'exécution continue au `case` suivant

`break;`
`continue;`
`return;`
`return x;`
	Sort du `while`, `do`, `for` ou `switch` le plus interne
	Passe à l'itération suivante du `while`, `do` ou `for`
	Sort de la fonction sans valeur de retour
	Sort de la fonction et retourne x

`throw a;`
`try { a; }`
`catch (T& t) { b; }`
`catch (...) { c; }`
	Lève une exception. Sort de toute fonction jusqu'à ce qu'elle soit attrapée
	Évalue a et contrôle ses exceptions
	Si a lève une exception de type T, évalue b
	Si a lève une autre exception, évalue c

Fonctions

`double f(int x, short s);`	Déclare la fonction f avec 2 paramètres `int` et `short`, retournant un `double`
`int f(int x) { instructions; }`	Définit une fonction f. Portée globale
`void f();`	Déclare f sans paramètre ni valeur de retour
`void f(int a = 42);`	`f()` appelle `f(42)`
`void f(int a, int& b, const int& c);`	Paramètres passés par valeur, référence ou référence constante
`void f(int t1[], const int t2[]);`	Tableaux passés modifiables ou constants
`T operator+(T lhs, T rhs);`	`a + b` de type T appelle `operator+(a, b)`
`T operator-(T x);`	`-a` de type T appelle `operator-(a)`
`T& operator++(T& i);`	`++i` retourne une référence à i
`T operator++(T& i, int);`	`i++` retourne la valeur précédente de i
`ostream& operator<<(ostream& o, const T& t);`	Surcharge `<<` pour l'affichage

Paramètres et valeurs de retour peuvent être de tout type. Toute fonction doit être déclarée ou définie avant utilisation. Elle peut être déclarée avant et définie ensuite.

Un programme consiste en un ensemble de déclarations de variables et de définitions de fonctions globales (éventuellement dans plusieurs fichiers), dont exactement une doit être

```
int main() { instructions... }
int main(int argc, char* argv[]) { instructions... }
```

`argv` étant un tableau de `argc` chaînes de caractères contenant les arguments de la ligne de commande. Par convention, `main` retourne 0 en cas de succès, 1 ou plus en cas d'erreur.

Chaque fichier `.cpp` doit contenir ou inclure la déclaration de toute fonction qu'il utilise. Chaque fonction utilisée doit être définie une et une seule fois dans l'ensemble des fichiers `.cpp`.

Surcharge. Elle est résolue via les paramètres selon l'ordre suivant : type exact, promotion numérique vers `int` ou `double`, conversions de type. Si elle est appelable, une référence variable est préférée à une référence constante. Si la fonction a plusieurs paramètres, l'ensemble des fonctions candidates est établi séparément pour chaque paramètre et l'intersection de ces ensembles est considérée. Si la résolution ne peut choisir une unique fonction, l'appel est ambigu et ne compile pas.

<cassert>

Macro d'arrêt du programme si une condition n'est pas remplie

`assert(e);`	Si e est faux, appelle `abort()`
`#define NDEBUG`	`assert` est désactivé si cette macro est définie avant `#include <cassert>`

Expressions

Les opérateurs sont groupés par ordre de précédence décroissante. Les opérateurs unaires, l'opérateur conditionnel et les affectations s'associent de droite à gauche, les autres de gauche à droite. La précédence n'affecte pas l'ordre d'évaluation, qui est non spécifié (sauf pour `&&`, `||`, `?:` et `,`).

1	`T::x`	Nom x défini dans la classe T
	`N::x`	Nom x défini dans l'espace de noms N
	`::x`	Nom global x
2	`t.x`	Membre x de l'objet (classe ou structure) t
	`p->x`	Membre x de l'objet pointé ou itéré par p
	`a[i]`	$(i+1)^{ème}$ élément de a, les indices commencent à 0
	`f(x, y)`	Appel à la fonction f avec les arguments x et y
	`T(x, y)`	Objet de classe T construit avec les arguments x et y
	`x++`	Incrémente x, retourne une copie du x original
	`x--`	Décrémente x, retourne une copie du x original
	`dynamic_cast<T>(x)`	Conversion au type T, vérifiée à l'exécution
	`static_cast<T>(x)`	Conversion au type T, non vérifiée
	`reinterpret_cast<T>(x)`	Interprète les bits de x comme étant de type T
	`const_cast<T>(x)`	Ajoute ou retire `const` au type de x
3	`sizeof(T)`	Nombre de bytes pour stocker un objet de type T
	`sizeof x`	Idem pour l'objet x, qui n'est pas évalué
	`++x`	Incrémente x, retourne une référence à x
	`--x`	Décrémente x, retourne une référence à x
	`-x`	Moins unaire
	`+x`	Plus unaire. Sans effet sauf promotion numérique
	`!x, not x`	Négation : vrai si x est faux, faux sinon
	`~x, compl x`	Complément bit à bit
	`*x`	Déréférence le pointeur / l'itérateur x
	`&x`	Adresse en mémoire de la variable x
	`(T) x`	Conversion au type T (style C, déconseillé)
4	`x * y`	Multiplication
	`x / y`	Division (réelle ou entière selon les arguments)
	`x % y`	Reste (x, y entiers ; x % y de même signe que x)
5	`x + y`	Addition
	`x - y`	Soustraction
6	`x << y`	Décalage des bits à gauche, retourne $x \cdot 2^y$
	`x >> y`	Décalage des bits à droite, retourne $x/2^y$. `<<` et `>>` sont surchargés pour `ostream&` et `istream&`
7	`x < y`	Strictement plus petit
	`x <= y`	Plus petit ou égal. Équivalent à `not (y < x)`
	`x > y`	Strictement plus grand. Équivalent à `(y < x)`
	`x >= y`	Plus grand ou égal. Équivalent à `not (x < y)`
8	`x == y`	Égalité
	`x != y`	Inégalité
9	`x & y, x bitand y`	Et appliqué bit par bit
10	`x ^ y, x xor y`	Ou exclusif appliqué bit par bit
11	`x | y, x bitor y`	Ou appliqué bit par bit
12	`x && y, x and y`	Et logique. y n'est évalué que si x est vrai
13	`x || y, x or y`	Ou logique. y n'est évalué que si x est faux
14	`x ? y : z`	y si x est vrai, z sinon. L'autre n'est pas évalué
	`x = y`	Affecte la valeur de y à x, retourne une référence à x
	`x += y`	`x = x + y` ; aussi `-=`, `*=`, `/=`, `%=`, `<<=`, `>>=`, `&=`, `|=`, `^=`
	`throw x`	Lève l'exception x
15	`x, y`	Évalue x puis y. Retourne y

Classes

```
class T {
```
Déclare / définit un nouveau type T

```
private:
```
Accessible seulement depuis les méthodes de T

```
public:
```
Accessible depuis partout

```
  int x;
```
Attribut

```
  void f(int x);
```
Méthode, aussi appelée fonction membre

```
  void g() { }
```
Méthode définie en ligne

```
  void h() const;
```
Méthode ne modifiant pas les attributs

```
  int operator+(int y);
```
`t + y` appelle `t.operator+(y)`

```
  int operator-();
```
`-t` appelle `t.operator-()`

```
  T() : x(1) {}
```
Constructeur avec liste d'initialisation

```
  T(const T& t) : x(t.x) {}
```
Constructeur de copie

```
  T& operator=(const T& t) {
    x = t.x; return *this; }
```
Opérateur d'affectation

```
  ~T();
```
Destructeur

```
  explicit T(int a);
```
Permet `T t(3);` mais pas `T t = 3;`

```
  operator int() const;
```
Conversion vers le type `int` ; permet `int(t)`

```
  friend void i();
```
La fonction `i()` a accès aux membres privés

```
  friend class U;
```
Les méthodes de U ont accès aux membres privés

```
  static int y;
```
Attribut partagé par tous les objets de type T

```
  static void k();
```
Méthode `T::k()`, a accès à y mais pas à x

```
  class Z {};
```
Classe imbriquée `T::Z`

```
  using V = int;
```
`T::V` est synonyme de `int`

```
};
```

```
void T::f(int x) {
  this->x = x;
}
```
Définition de la méthode f de T. `this` est l'adresse de l'objet lui-même : `this->x` est l'attribut, x le paramètre

```
int T::y = 2;
```
Initialisation d'un attribut statique

```
T::k();
```
Appel à la méthode statique k de T

```
T(const T&) = delete;
```
Supprime une fonction (ici, le constructeur de copie)

```
T() = default;
```
Demande la version générée par le compilateur

S'ils ne sont pas explicitement définis ou supprimés (`= delete`), toute classe a un constructeur de copie et un opérateur d'affectation par défaut qui copient tous les attributs. Si aucun constructeur n'est explicitement défini, il y a aussi un constructeur par défaut sans paramètre.

Exceptions et fin de programme

```
void f();
```
f peut lever des exceptions

```
void g() noexcept;
```
g ne lève pas d'exception

```
void h() noexcept(c);
```
h ne lève pas d'exception si c est vrai

```
return 0;
```
Sortie normale du programme depuis `main()`

```
exit(0);
```
Sortie normale du programme depuis n'importe où ; ne détruit pas les objets automatiques · `<cstdlib>`

```
atexit(f); atexit(g);
```
`g()` puis `f()` seront appelées en sortie normale · `<cstdlib>`

```
abort();
```
Sortie anormale, vers le débogueur · `<cstdlib>`

```
terminate();
```
Appelée quand une exception n'est pas capturée. Appelle `abort()` (par défaut), pas `exit()` · `<exception>`

```
set_terminate(f);
```
`f()` sera appelée par `terminate()` à la place du gestionnaire par défaut (`abort`) · `<exception>`

<stdexcept>

Met à disposition des classes dérivées de `std::exception` pour signaler des erreurs spécifiques.

```
throw logic_error(
  "Pas d'opérateur < disponible");
```
Lève une exception standard

```
try { f(); }
```
Bloc protégé

```
catch (const logic_error& e) {
  cerr << e.what(); }
```
Traitement d'une exception

```
catch (...) { cerr << "Autre"; }
```
Attrape tous les types d'exceptions

```
invalid_argument("constructeur faux")
```
Erreur spécifique, dérive de `logic_error`

```
length_error("Dépassement taille max")
```
Dérive de `logic_error`

```
out_of_range("Indices hors tableau")
```
Dérive de `logic_error`

```
range_error("Valeur trop grande")
```
Dérive de `runtime_error`

```
overflow_error("Dépassement du type int")
```
Dérive de `runtime_error`. Jamais levée automatiquement : un dépassement sur `int` est un comportement indéfini

```
bad_alloc()
```
Levée par `new` (mémoire insuffisante), dérive de `exception` · `<new>`

<fstream>

`fstream fs("f.txt", ios::in | ios::out);`	Crée fs et ouvre f.txt en lecture et écriture
`ios::app, ios::trunc`	Ajout à la fin, écrasement
`fs >> x >> y;`	Lit x et y depuis le flux fs
`fs << x << y;`	Écrit x et y dans le flux fs
`fs.close();`	Fermeture du fichier
`fs.open("f.txt", ios::in | ios::out);`	Connecte le flux fs au fichier f.txt
`ifstream is("f.txt");`	Création du flux is (en lecture)
`c = is.get();`	Lit 1 `char`
`is.get(c);`	Lit 1 `char`. Retourne is
`is.unget();`	Remet le dernier caractère lu
`getline(is, string& s, char d = '\n');`	Lit jusqu'à d, stocke le résultat dans s
`is.ignore(size_t n = 1, int delim = EOF);`	Ignore au maximum n `char`, ou jusqu'à delim
`ofstream os("f.txt");`	Création du flux os (en écriture)
`os.put(c);`	Écrit le caractère c dans le flux os
`os.flush();`	Force l'écriture

<iostream>

`cin >> x >> y;`	Lit x et y depuis l'entrée standard
`cout << "x=" << 3 << flush;`	Affiche x=3 et force l'impression
`cerr << x << y << endl;`	Affiche x et y, passe à la ligne, sans tampon
`c = cin.get();`	Lit 1 `char`
`if (cin)`	Teste si `!cin.fail()`
`cin.clear();`	Remet cin à l'état good
`cin.ignore(n, '\n');`	Ignore n caractères ou jusqu'au saut de ligne
`cin.ignore(numeric_limits<streamsize>::max(), '\n');`	Vide la ligne en cours

<iomanip> et <ios>

`cout << setw(6) << setprecision(3) << setfill('*') << 31.41592;`	`**31.4`
`cout << fixed << setprecision(3) << 31.41592;`	`31.416`
`cout << scientific << setprecision(3) << 31.41592;`	`3.142e+01`
`cout << setfill('*') << left << setw(4) << +2;`	`2***`
`cout << setfill('*') << right << setw(4) << -2;`	`**-2`
`cout << showpos << setfill('*') << internal << setw(4) << +2;`	`+**2`
`cout << oct << 16 << ' ' << dec << 16 << ' ' << hex << 16;`	`20 16 10`

<cmath>

`sin(x); cos(x); tan(x);`	Fonctions trigonométriques, x en radians
`asin(x); acos(x); atan(x);`	Fonctions trigonométriques inverses
`sinh(x); cosh(x); tanh(x);`	Fonctions hyperboliques
`exp(x); log(x); log10(x); log2(x);`	e^x, logarithmes en base e, 10, 2
`pow(x, y); sqrt(x); hypot(x, y);`	x^y (imprécis sur entiers), $x^{1/2}$, $(x^2+y^2)^{1/2}$
`ceil(x); floor(x); trunc(x);`	Entier supérieur / inférieur / tronqué
`round(x); round(1.5); round(-3.5);`	Entier le plus proche / 2 / -4
`fabs(x); fmod(x, y);`	Valeur absolue, x modulo y

<limits>

`numeric_limits<T>::max();`	Plus grande valeur du type T
`numeric_limits<T>::min();`	Plus petite valeur pour un entier ; plus petite valeur **positive** pour un réel : 1.17549e-38 pour `float`
`numeric_limits<T>::lowest();`	Plus petite valeur : -3.40282e+38 pour `float`
`numeric_limits<T>::digits10;`	Chiffres significatifs : 6 pour `float`
`numeric_limits<T>::epsilon();`	Précision relative : 1.2e-07 pour `float`

<cctype>

`isalpha(c); isalnum(c);`	c est une lettre ? Une lettre ou un chiffre ?
`isdigit(c); isxdigit(c);`	c est un chiffre décimal ? Hexadécimal ?
`isblank(c); isspace(c);`	`' '` ou `'\t'` ? Idem, ou `'\n'`, `'\v'`, `'\r'`, `'\f'` ?
`ispunct(c); iscntrl(c);`	Ponctuation ? Contrôle (< 32 ou 127) ?
`isprint(c); isgraph(c);`	Pas contrôle ? Comme isprint, sauf `' '` ?
`islower(c); isupper(c);`	c est minuscule ? Majuscule ?
`tolower(c); toupper(c);`	Convertit en minuscule / majuscule

<array>

Tableaux de taille connue à la compilation

```
array<int, 4> a;
array<int, 4> b(a);
array<int, 4> c = {1, 2};
```

Tableau de 4 entiers {?, ?, ?, ?}
b est une copie de a
Tableau {1, 2, 0, 0}

```
a.size();
a.data();
a.empty();
a[2];
a.at(3);
a.back();
a.front();
```

Nombre d'éléments de a
Pointeur vers le premier élément
`a.size() == 0`
Référence au troisième élément de a
`a[3]`, lève `out_of_range` si `3 >= a.size()`
`a[a.size() - 1]`
`a[0]`

```
for (int e : a) cout << e;
for (int& e : a) e = 0;
a.fill(42);
a = b;
a == b; a < b; ...
```

Parcours de tout a en lecture
Parcours permettant de modifier a
`for (int& e : a) e = 42;`
Copie tout b dans a
Comparaison lexicographique

<vector>

Comme array (sauf fill), mais de taille et capacité variables, donc aussi...

```
vector<int> v(3);
vector<int> x(n, 1);
```

Tableau de 3 entiers {0, 0, 0}
x rempli de n fois la valeur 1

```
v.push_back(7);
v.pop_back();
v.assign(...);
v.resize(n, 1);
v.clear();
```

Ajoute l'élément 7 à la fin. Incrémente `v.size()`
Supprime le dernier élément. Décrémente `v.size()`
`v = vector<int>(...);`
Modifie la taille, remplit avec 1 si elle augmente
`v.resize(0);`

```
v.capacity();
v.reserve(n);
v.shrink_to_fit();
```

Nombre d'emplacements mémoire réservés
Augmente la capacité si `n > v.capacity()`
Réduit la capacité à `v.size()`

<string>

Instanciation de `basic_string<char>`, comme `vector<char>`, mais avec des fonctions membres prenant en paramètre des `size_t` spécifiant position et longueur plutôt que des itérateurs, et aussi...

```
string s1, s2 = "hello", s3(4, 'a');
string s4(s2, 1, 2), s5(s2, 3);
string s6("hello", 4);
```

"", "hello", "aaaa"
"el", "lo" : sous-chaînes de s2
"hell" : 4 premiers `char` du `const char[]`

```
s1.length();
s1 += s2 + ' ' + "world";
s1.append(s2, pos, len);
s1.append("abc", 2); s1.append(4, 'x');
s1.compare(s2);
s1.compare(pos, len, s2, subpos, sublen);
s.substr(m, n);
```

Synonyme de `s1.size()`
Concaténations
Concaténation d'une sous-chaîne
Concaténation de "ab" ; de "xxxx"
< 0 si s1 < s2, 0 si s1 == s2, > 0 sinon
Comparaison de 2 sous-chaînes
Sous-chaîne de n `char` commençant à `s[m]`

```
size_t i = s.find(x, pos);
size_t i = s.rfind(x, pos);
size_t i = s.find_first_of(x, pos);
size_t i = s.find_last_of(x, pos);
string::npos;
s1.replace(pos, len, "xxx", 2);
to_string(3.14);
```

Cherche depuis l'indice pos, retourne `string::npos` si absent
Cherche de droite à gauche
Recherche parmi une liste de `char`
Idem, de droite à gauche
`size_t(-1)`
Remplace len `char` depuis pos par "xx"
Convertit une valeur numérique en string au format par défaut. Ici "3.140000" (jusqu'à C++23)